Structured API - PySpark

Wednesday, 2 September 2026 8:22 PM

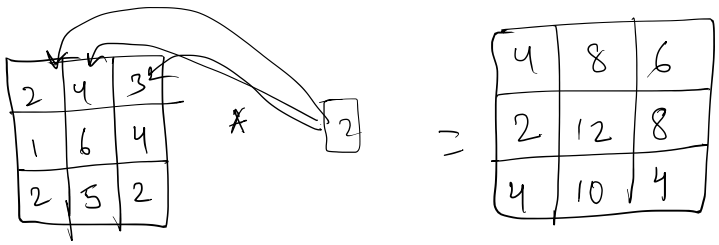
RDD: Resilient Distributed Datasets.

- RDDs are imperative: Tell 'how to do things'
  ↓
  We have to write exact step by step instructions.
- We need functional programming.

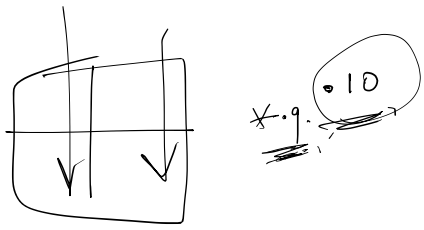
DataFrames:

- Dataframes are declarative.
- Tell Pyspark 'what' to do
- Row & column Structure.

Feature/Aspect	RDD	Dataframe.
Data Abstraction.	Distributed collection of Python/ Scala/ Java Objects	Distributed table with row and column structure.
Structure Awareness	No schema, data is opaque.	Schema aware. column name, data types, nullability.
Programming Style	Imperative	Declarative.
Ease of analytics	Low	High. SQL style queries and high level API supported.
Performance	Slower for analytical queries	Faster for analytical queries.
Accessibility	Engineers	Engineers + Analysts.
Row/Column Operation	Row level transformation.	Column level transformation.
Best suited for?	Custom processing / legacy pipelines.	Structured analysis / SQL workflows
Join logic	Manual join logics.	Optimized joins (boardcasting,



Broadcasting.



rdd.map(lambda x,y: x+10)

[1, 2, 3, 4, 5]
x x

11, 12, 13, 14, 15.

reduce by key

rdd = [(cat, 1), (dog, 2), (horse, 3), (cat, 10), (dog, 1)]

rdd.reduceByKey(lambda x,y: x+y).

① Grouping.

cat → 1, 10, 2, 4
x y
dog → 2, 1
horse → 3.

② Reduce.

cat → 1+10 → 11+2 → 13+4
dog → 2+1
horse → 3

(2) Final output.

[(cat, 11), (dog, 3), (horse, 3)]

```
def sum(x,y):
    return x+y
```

① df = spark.read('booth.csv')

↳ df.groupby('col1').agg(col2: sum).

↳

↳

↳

→ filtered_df = ↳ filter.

→ Spark.write('filtered_df.csv')

Storage.

RDD Dataset.